UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 05 – Transacciones distribuidas

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Primer cuatrimestre 2026

Equipo docente

11 de agosto de 2026

1 Introducción

Las transacciones permiten agrupar operaciones de forma que el sistema preserve invariantes incluso bajo concurrencia.

En sistemas distribuidos, esto implica:

- múltiples nodos
- múltiples copias
- latencias y fallas

2 Modelo de transacciones

Una transacción T es una secuencia:

$$T = \{r(x), w(x), commit\}$$

Conflictos:

- Read-Write (RW)
- Write-Read (WR)
- Write-Write (WW)

3 Serializabilidad

3.1 Definición

Una ejecución es serializable si existe una ejecución secuencial equivalente.

3.2 Grafo de precedencia

Nodos: transacciones Aristas: dependencias por conflicto

Teorema El schedule es serializable el grafo es acíclico.

4 Anomalías

4.1 Dirty read

Lectura de datos no confirmados.

4.2 Non-repeatable read

Lecturas inconsistentes dentro de la misma transacción.

4.3 Phantom

Cambios en conjuntos de resultados.

4.4 Write skew

Dos transacciones leen el mismo estado y escriben sin conflicto directo.

Ejemplo Dos médicos verifican guardia:

- ambos leen que hay otro médico
- ambos se retiran

Resultado: ningún médico.

5 Snapshot Isolation

Cada transacción observa un snapshot consistente.

5.1 Propiedad

Evita:

- dirty read
- non-repeatable read

No evita:

- write skew

6 Commit distribuido

6.1 2PC

Fases:

1. Prepare
2. Commit

6.2 Problema

Bloqueo si el coordinador falla.

7 3PC

Introduce fase:

- pre-commit

Reduce bloqueo pero requiere sincronía.

8 Relación con consenso

- consenso $\Rightarrow$ orden global
- transacciones $\Rightarrow$ consistencia lógica

9 Conclusión

La serializabilidad garantiza correctitud, pero requiere coordinación fuerte y tiene costo.

10 Síntesis razonada de la clase

Esta unidad debe leerse como parte de una cadena de decisiones de diseño y no como un catálogo de mecanismos. Para estudiar el tema con profundidad conviene reconstruir cuatro preguntas: *qué problema aparece*, *qué garantía se desea*, *qué mecanismo la aproxima* y *qué costo introduce*. En cada ejemplo debe identificarse además el modelo de falla implícito.

Una respuesta técnicamente madura no termina al nombrar un algoritmo o patrón: declara sus supuestos, la propiedad que preserva y el escenario en el que deja de ser suficiente.

11 Bibliografía guiada y ruta de profundización

Para profundizar esta clase se recomienda: Kleppmann, cap. 7; Gray y Reuter; Berenson et al. sobre isolation levels.

La lectura debe hacerse con preguntas concretas: (1) ¿qué modelo de sistema asume el autor?, (2) ¿qué propiedad demuestra o persigue?, (3) ¿qué falla o anomalía motiva el mecanismo?, y (4) ¿qué trade-off queda abierto?

12 Conexión con el Trabajo Final Integrador

El grupo debe señalar qué parte de su sistema utiliza los conceptos de esta unidad, justificar por qué son necesarios y documentar una alternativa descartada. Cuando el contenido todavía no corresponda a la etapa vigente, debe registrarse como hipótesis o decisión pendiente.

13 Autoevaluación conceptual

1. ¿Cuál es el problema distribuido concreto que motiva esta clase?
2. ¿Qué propiedad de seguridad y qué propiedad de progreso están en juego?
3. ¿Qué supuesto de red, tiempo o falla resulta decisivo?
4. ¿Qué trade-off aceptaría en un sistema real y cuál no aceptaría en un dominio crítico?
5. ¿Cómo observaría experimentalmente que la solución se comporta como espera?

14 Profundización autocontenida v9

14.1 Problema, límite e invariante

El núcleo de esta clase es ACID, serializabilidad, grafos de precedencia, anomalías, Snapshot Isolation, 2PC y sagas. Para estudiarlo de manera autónoma conviene separar tres planos. Primero, el *problema observable*: qué comportamiento incorrecto puede ver un cliente o un operador. Segundo, el *límite del modelo*: qué información, sincronía o coordinación no está disponible. Tercero, el *invariante*: qué propiedad no debe violarse aunque el sistema pierda progreso temporalmente.

El modelo de fallas relevante incluye concurrencia, aborto parcial, coordinador caído y violación de invariantes. Una solución debe describirse como protocolo y no solo como nombre de tecnología: actores, estados, mensajes, condición de éxito, condición de aborto y estado visible después de una interrupción. La evidencia esperada es schedule, grafo de precedencia, estado prepared y resultado observable.

14.2 Método de análisis

Ante un caso nuevo, siga esta secuencia:

1. Identifique actores, dato o recurso compartido y frontera de confianza.
2. Escriba una ejecución nominal y una ejecución adversarial.
3. Distinga seguridad (lo malo no ocurre) de progreso (algo bueno finalmente ocurre).
4. Declare qué supuesto permite avanzar: mayoría, deadline, sincronía parcial, operación idempotente, almacenamiento durable u otro.
5. Compare una alternativa más simple y una más fuerte; registre qué métrica o experimento haría cambiar la decisión.

14.3 Caso transferible

Considere una plataforma de reservas que recibe operaciones duplicadas, pierde conectividad entre regiones y debe sostener un camino crítico sin ocultar el estado real al usuario. Aplique el mecanismo de esta clase a una única decisión concreta. Una respuesta madura no intenta resolver todo: delimita la operación, formula la garantía y explica la degradación esperada cuando el supuesto central deja de cumplirse.

14.4 Errores de razonamiento frecuentes

- confundir una propiedad con el producto que suele implementarla;
- describir el camino feliz sin recorrer una falla intermedia;
- afirmar “exactly-once”, “alta disponibilidad” o “consistencia” sin definir alcance observable;
- medir solo throughput aunque la hipótesis trate sobre semántica o recuperación;
- presentar la complejidad añadida como beneficio sin compararla con la alternativa más simple.

15 Lectura guiada

Kleppmann, capítulo 7; Berenson et al. sobre isolation levels; Gray y Reuter.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría. Estas cinco anotaciones convierten la bibliografía en una herramienta de diseño y no en una lista para memorizar.

16 Aplicación al Trabajo Final Integrador

El equipo debe producir un ADR breve con la decisión asociada a esta clase. Debe contener: contexto, dos alternativas, supuesto decisivo, garantía elegida, trade-off, evidencia pendiente y fecha de revisión.

La evidencia mínima esperada para esta unidad es: schedule, grafo de precedencia, estado prepared y resultado observable.

17 Autoevaluación avanzada

1. ¿Puedo explicar la clase sin nombrar primero una tecnología?
2. ¿Puedo construir una ejecución que viole la garantía si el mecanismo se configura mal?
3. ¿Puedo distinguir seguridad, progreso y experiencia del usuario?
4. ¿Puedo proponer una medición que valide la propiedad, no solo el rendimiento?
5. ¿Puedo defender cuándo una solución más simple sería suficiente?